

CAN Motor Control for Legged-Robot Joints

Physics, Mathematics, and Code Reference

Project: `can_motor_control`

June 19, 2026

Contents

1	Introduction and System Overview	3
2	CAN Bus Communication	3
2.1	Physical and data-link layer	3
2.2	Frame structure	3
2.3	Integer encoding: little-endian two's complement	3
2.4	Command set	4
2.5	Request-response and timing	4
3	Units, Gains, and Physical Conversions	4
3.1	Gear reduction	5
3.2	Position	5
3.3	Velocity — command and feedback	5
3.4	Encoder	5
3.5	Torque and current	5
3.6	Sign conventions	6
4	Worked Numerical Examples	6
4.1	Encoding a position command	6
4.2	Negative values (two's complement)	6
4.3	Decoding a status-2 reply	6
4.4	Velocity and torque commands	7
5	Motor Control Modes	7
5.1	Background: brushless torque and FOC	7
5.2	Torque (current) loop — <code>0xA1</code>	7
5.3	Speed loop — <code>0xA2</code>	7
5.4	Position loop — <code>0xA3/0xA4</code>	7
6	Leg Kinematics and Dynamics	7
6.1	Forward kinematics	8
6.2	Jacobian and static force mapping	8
6.3	Joint-space dynamics	8
6.4	Cartesian (foot-space) impedance — the next step	8

7	A Short Control-Theory Primer	8
7.1	Feedback and the PID family	9
7.2	Feedforward	9
7.3	Cascade control	9
7.4	Stability, delay, and sampling	9
8	Trajectory-Following Control	9
8.1	Why not “MIT mode”?	9
8.2	Velocity feedforward + position PD	10
8.3	Trajectory generation	10
8.4	Encoder unwrapping	10
8.5	Discretisation and loop rate	10
9	Position vs. Torque Control	11
9.1	The trade-off	11
9.2	Software joint impedance	11
9.3	Torque-to-current conversion	11
9.4	Gravity feedforward	12
9.5	When to use which	12
10	Code Architecture	12
10.1	File map	12
10.2	The LKMotor class	12
10.3	Walkthrough: <code>trajectory_follow.py</code>	12
10.4	Walkthrough: <code>torque_impedance.py</code>	13
11	Practical Notes, Limitations, and Tuning	13
12	Suggested Next Steps	14
A	Symbol Reference	14
B	Glossary	14

1 Introduction and System Overview

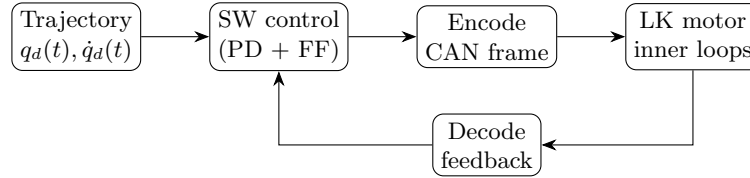
This document explains the theory and implementation behind `can_motor_control`, a Python package that drives **LK Motor** brushless servo actuators over a **CAN bus** for use as the joints of a legged robot. The target application is a planar leg with two joints:

- **m1** — hip / thigh joint (*datui*), positive direction counter-clockwise (`dir_1 = +1`);
- **m2** — knee / shin joint (*xiaotui*), positive direction clockwise (`dir_2 = -1`).

The software has three layers:

1. **Transport & protocol** — `motor_library.py`: the `LKMotor` class encodes/decodes CAN frames and exposes the motor's control and status commands.
2. **Calibration** — `config.py`: the gains that convert between raw motor units and physical engineering units ($^{\circ}$, $^{\circ}/s$, Nm).
3. **Application** — `main.py`, `demo_motors.py`, `trajectory_follow.py`: position tests and the trajectory-tracking controller.

The signal chain from a desired joint angle to motion is:



2 CAN Bus Communication

2.1 Physical and data-link layer

CAN (Controller Area Network) is a differential, multi-drop serial bus. Here it is accessed through Linux **SocketCAN** using the `python-can` library, with a USB-CAN adapter presented as interface `can0`. The bus runs at 1 Mbit s^{-1} .

Important: for SocketCAN the bit rate is fixed at the *link* level, not by software:

```
sudo ip link set can0 up type can bitrate 1000000
```

The `bitrate=1.000.000` keyword passed to `can.Bus` is ignored by the SocketCAN backend.

2.2 Frame structure

Every command is a standard (11-bit identifier) data frame with exactly **8 data bytes**. For a single-motor command/reply the identifier is

$$\text{arbitration_id} = 0x140 + \text{motor_id}. \quad (1)$$

The motor replies on the *same* identifier. Byte 0 (`DATA[0]`) is always the **command byte**; bytes 1–7 carry parameters or status, laid out per command:

$$\underbrace{\text{DATA}[0]}_{\text{command}} \quad \underbrace{\text{DATA}[1] \cdots \text{DATA}[7]}_{7 \text{ payload bytes}}$$

2.3 Integer encoding: little-endian two's complement

Multi-byte integers are transmitted **least-significant byte first** (little-endian). Signed values use two's complement over the chosen byte width.

Encoding For a value v stored in n bytes, the helper first maps negatives into the unsigned range,

$$\tilde{v} = \begin{cases} v, & v \geq 0, \\ v + 2^{8n}, & v < 0, \end{cases} \quad (2)$$

then extracts byte i (for $i = 0, \dots, n-1$) as

$$b_i = \lfloor \tilde{v} / 2^{8i} \rfloor \bmod 256 = (\tilde{v} \gg 8i) \& 0xFF. \quad (3)$$

Decoding Given bytes $b_0, \dots, b_{n-1}$ the magnitude is reconstructed and the sign restored from the most-significant bit of the top byte:

$$v = \sum_{i=0}^{n-1} b_i 2^{8i} - \begin{cases} 2^{8n}, & b_{n-1} \geq 128 \text{ (MSB set)}, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

These are implemented as `_decimal_to_byte` and `_byte_to_decimal` and underpin every parameter and every status field.

2.4 Command set

Cmd	Method	Function
0x88	<code>motor_run</code>	Enable / power the motor.
0x81	<code>motor_stop</code>	Stop motion, keep state.
0x80	<code>motor_shutdown</code>	Power off, clear turns/commands.
0x9A	<code>read_motor_status_1</code>	Temp, voltage, current, state, errors.
0x9B	<code>clear_error_flags</code>	Clear error state.
0x9C	<code>read_motor_status_2</code>	Temp, i_q , speed, encoder.
0x9D	<code>read_motor_status_3</code>	Temp, phase currents A, B, C .
0xA0	<code>open_loop_control</code>	Open-loop power (MS series).
0xA1	<code>torque_loop_control</code>	Current/torque closed loop.
0xA2	<code>speed_loop_control</code>	Velocity closed loop.
0xA3/0xA4	<code>multi_turn_position_control</code>	Absolute multi-turn position ($\pm$ speed cap).
0xA5/0xA6	<code>single_turn_position_control</code>	Absolute single-turn position.
0xA7/0xA8	<code>incremental_position_control</code>	Relative position step.
0x90	<code>read_encoder_data</code>	Encoder value, raw, offset.
0x92	<code>read_multi_turn_angle</code>	Multi-turn angle (0.01° /LSB).
0x94	<code>read_single_turn_angle</code>	Single-turn angle (0–35999).
0x19	<code>set_zero_position</code>	Persist current pos. as zero (flash).
0x95	<code>set_position_to_angle</code>	Reassign current pos. (RAM).

2.5 Request–response and timing

Each method *sends* a frame and *blocks* waiting for the matching reply (`_receive_response`, default timeout 0.5 s, polling in 50 ms slices). The call returns `None` on timeout. This synchronous round-trip is the dominant factor limiting control-loop rate (Section 8.5).

3 Units, Gains, and Physical Conversions

The motor reports and accepts values in raw LSB units at the *motor* shaft. The robot cares about the *output* shaft (after the gearbox), in degrees, $^\circ$ /s, and newton-metres. `config.py` stores the conversion gains.

3.1 Gear reduction

The actuator has a reduction such that

$$\theta_{\text{out}} = r_g \theta_{\text{motor}}, \quad r_g = 0.1, \quad (5)$$

i.e. a 10 : 1 reduction (the output shaft turns one tenth of the motor shaft, with $10\times$ the torque). This factor of 0.1 appears inside every gain.

3.2 Position

The position command (0xA4) uses $0.01^\circ/\text{LSB}$ at the motor shaft. To move the *output* by θ degrees:

$$N_{\text{cmd}} = \underbrace{\frac{\theta}{r_g}}_{\text{motor deg}} \cdot \underbrace{\frac{1}{0.01^\circ \text{LSB}^{-1}}}_{=100} = \frac{\theta}{0.1} \cdot 100 = \theta \cdot 1000 \equiv \theta \cdot \text{pos_gain}. \quad (6)$$

Hence $\boxed{\text{pos_gain} = 1000 \text{ LSB}/^\circ_{\text{out}}}$. Inversely, an output angle is recovered from a position-loop count by dividing by `pos_gain`.

3.3 Velocity — command and feedback

Command (speed loop, 0xA2). Unit $0.01 \text{ dps LSB}^{-1}$ at the motor. By the same reasoning as position,

$$N_{\dot{\theta}} = \dot{\theta} \cdot \frac{1}{r_g} \cdot 100 = \dot{\theta} \cdot 1000 \equiv \dot{\theta} \cdot \text{vel_gain}, \quad \text{vel_gain} = 1000. \quad (7)$$

Feedback (status 2). Speed is reported at 1 dps LSB^{-1} at the *motor* shaft, so the output velocity is

$$\dot{\theta}_{\text{out}} = \frac{\text{spd_raw}}{\text{vel_state_gain}}, \quad \text{vel_state_gain} = \frac{1}{r_g} = 10. \quad (8)$$

3.4 Encoder

A **single-turn** absolute encoder reports `enc` $\in [0, 65535]$ (16-bit) for one revolution of the output:

$$\theta_{\text{enc}} = \text{enc} \cdot \text{encoder_gain}, \quad \text{encoder_gain} = \frac{360}{65535}^\circ \text{LSB}^{-1}. \quad (9)$$

Because it is single-turn it *wraps* at 360° ; multi-turn motion must be reconstructed in software (Section 8.4).

3.5 Torque and current

The motor returns the quadrature current command i_q (`int16`). Output torque is obtained through a single lumped calibration constant:

$$\tau_{\text{out}} = \frac{\text{iq_raw}}{\text{torque_gain}}, \quad \text{torque_gain} = 206.04 \frac{\text{LSB}}{\text{N m}}. \quad (10)$$

Physically this constant bundles the current LSB scale ($\frac{33}{4096}$ or $\frac{66}{4096}$ A/LSB depending on model), the motor torque constant K_t , the gear ratio $1/r_g$, and drivetrain efficiency η :

$$\tau_{\text{out}} = \text{iq_raw} \cdot (\text{A/LSB}) \cdot K_t \cdot \frac{1}{r_g} \cdot \eta \implies \frac{1}{\text{torque_gain}} = (\text{A/LSB}) K_t \frac{\eta}{r_g}. \quad (11)$$

3.6 Sign conventions

Each joint has a direction multiplier $d \in \{+1, -1\}$ (`dir_1`, `dir_2`) chosen so that the robot's positive joint direction matches the desired convention. It is applied symmetrically: *multiply when commanding, multiply when reading*. For a physical quantity x and its raw counterpart x_{raw} ,

$$x_{\text{cmd,raw}} = \text{gain} \cdot d \cdot x, \quad x_{\text{meas}} = d \cdot \frac{x_{\text{raw}}}{\text{gain}}. \quad (12)$$

Quantity	Gain (name)	Relation
Position cmd	<code>pos_gain</code> = 1000	$N = \theta \cdot d \cdot 1000$
Velocity cmd	<code>vel_gain</code> = 1000	$N = \dot{\theta} \cdot d \cdot 1000$
Velocity fb	<code>vel_state_gain</code> = 10	$\dot{\theta} = d \cdot \text{spd}/10$
Encoder	<code>encoder_gain</code> = 360/65535	$\theta = d \cdot \text{enc} \cdot g_e$
Torque	<code>torque_gain</code> = 206.04	$\tau = d \cdot \text{iq}/206.04$

4 Worked Numerical Examples

This section traces concrete numbers all the way to bytes, to make the encoding and the gains tangible.

4.1 Encoding a position command

Goal: move the *output* shaft of motor id 1 to 30° with a speed cap of 600 dps (motor-shaft units), using `multi_turn_position_control`.

1. **Angle to LSB:** $N = 30 \times \text{pos_gain} = 30 \times 1000 = 30000$ LSB.
2. **30000 in 4 little-endian bytes:** $30000 = 0x7530$, so bytes (low→high) are 0x30, 0x75, 0x00, 0x00.
3. **Speed 600 in 2 bytes:** $600 = 0x0258 \Rightarrow 0x58, 0x02$.
4. **Assemble the 0xA4 frame** (`DATA` = [0, speed(2), angle(4)] with the command byte prepended):

byte	0	1	2	3	4	5	6	7
value	0xA4	0x00	0x58	0x02	0x30	0x75	0x00	0x00
role	cmd	—	spd _{lo}	spd _{hi}	ang ₀	ang ₁	ang ₂	ang ₃

This frame is sent on arbitration id $0x140 + 1 = 0x141$.

4.2 Negative values (two's complement)

To command -30000 LSB in 4 bytes: add 2^{32} to get $4294937296 = 0xFFFF8AD0$, giving bytes 0xD0, 0x8A, 0xFF, 0xFF. On decode, the top byte $0xFF \geq 128$ flags a negative, so 2^{32} is subtracted back.

4.3 Decoding a status-2 reply

Suppose the reply payload is [0x9C, 0x1A, 0x12, 0x00, 0x64, 0x00, 0x00, 0x40]:

- temperature = $0x1A = 26^\circ\text{C}$;
- $i_q = 0x0012 = 18$ LSB $\Rightarrow \tau = 18/206.04 \approx 0.087$ N m;
- speed = $0x0064 = 100$ (motor dps) $\Rightarrow 100/10 = 10$ dps output;
- encoder = $0x4000 = 16384 \Rightarrow 16384 \times \frac{360}{65535} \approx 90.0^\circ$.

4.4 Velocity and torque commands

- **Velocity:** 60 dps output $\Rightarrow N = 60 \times \text{vel_gain} = 60000$ LSB (= 0xEA60).
- **Torque:** 2 N m $\Rightarrow i_q = 2 \times \text{torque_gain} = 412$ LSB (= 0x019C), well within the ± 2048 range.

5 Motor Control Modes

The driver implements three nested closed loops on-board; the host selects which one to command.

5.1 Background: brushless torque and FOC

The actuator is a three-phase brushless DC (BLDC / PMSM) motor. Its driver uses **field-oriented control (FOC)**: the three phase currents are transformed (Clarke + Park) into a rotating frame aligned with the rotor, giving a *direct* component i_d and a *quadrature* component i_q . Torque is produced almost entirely by i_q ,

$$\tau \approx K_t i_q, \quad i_d \approx 0, \quad (13)$$

so regulating i_q is regulating torque — which is why every higher loop ultimately outputs an i_q reference, and why `torque_gain` relates i_q to output torque.

5.2 Torque (current) loop — 0xA1

Commands $i_q \in [-2048, 2048]$. The driver regulates winding current via field-oriented control (FOC) so that the measured i_q tracks the command, producing torque $\tau \approx K_t i_q$. This is the innermost loop.

5.3 Speed loop — 0xA2

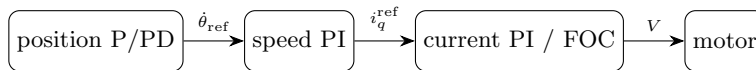
Commands an output velocity (in LSB, Section 3). Internally a PI controller drives the *torque* loop:

$$i_q^{\text{ref}} = K_p^v (\dot{\theta}_{\text{ref}} - \dot{\theta}) + K_i^v \int (\dot{\theta}_{\text{ref}} - \dot{\theta}) dt. \quad (14)$$

This is the loop used by the trajectory follower.

5.4 Position loop — 0xA3/0xA4

Commands an absolute multi-turn angle, with an optional maximum-speed cap (0xA4). Internally a position controller feeds the speed loop, which feeds the torque loop — a full **cascade**:



The `max_speed` argument bounds the inner velocity reference and is the simplest way to make a point-to-point move slower or faster.

6 Leg Kinematics and Dynamics

To interpret joint commands at the foot we model the leg as a planar 2R chain: hip angle q_1 , knee angle q_2 , link lengths ℓ_1 (thigh) and ℓ_2 (shin).

6.1 Forward kinematics

With angles measured from the $+x$ axis and the chain rooted at the hip, the foot position (x, y) is

$$x = \ell_1 \cos q_1 + \ell_2 \cos(q_1 + q_2), \quad (15)$$

$$y = \ell_1 \sin q_1 + \ell_2 \sin(q_1 + q_2). \quad (16)$$

6.2 Jacobian and static force mapping

Differentiating, the foot velocity relates to joint rates through the Jacobian $J(q)$:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = J(q) \dot{q}, \quad J = \begin{bmatrix} -\ell_1 s_1 - \ell_2 s_{12} & -\ell_2 s_{12} \\ \ell_1 c_1 + \ell_2 c_{12} & \ell_2 c_{12} \end{bmatrix}, \quad (17)$$

where $s_1 = \sin q_1$, $c_{12} = \cos(q_1 + q_2)$, etc. By the principle of virtual work, a desired ground-reaction / foot force $\mathbf{F}$ maps to joint torques as

$$\boxed{\boldsymbol{\tau} = J(q)^\top \mathbf{F}} \quad (18)$$

This is how a force at the foot (e.g. supporting body weight, or a desired push-off) is turned into the τ each joint must produce — and hence, via `torque_gain`, into an i_q command.

6.3 Joint-space dynamics

The leg obeys the manipulator equation

$$M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + g(q) = \boldsymbol{\tau}, \quad (19)$$

with inertia matrix M , Coriolis/centrifugal term C , and gravity g . A model-based controller would compute a feedforward $\boldsymbol{\tau}_{\text{ff}} = M\ddot{q}_d + C\dot{q}_d + g$ and add a PD term; the present software uses the motor's internal loops instead of an explicit inverse-dynamics model (see next section).

6.4 Cartesian (foot-space) impedance — the next step

The most useful capability for a leg is to control the *foot* rather than the individual joints. Define a desired foot position $\mathbf{x}_d$ and a virtual spring-damper in Cartesian space:

$$\mathbf{F} = K_x (\mathbf{x}_d - \mathbf{x}) + D_x (\dot{\mathbf{x}}_d - \dot{\mathbf{x}}), \quad (20)$$

where $\mathbf{x}$ comes from forward kinematics and $\dot{\mathbf{x}} = J\dot{q}$. Map that desired foot force to joint torques with the Jacobian transpose, adding gravity compensation:

$$\boxed{\boldsymbol{\tau} = J(q)^\top \mathbf{F} + g(q)} \quad (21)$$

and send each τ through the torque loop exactly as in `torque_impedance.py`. This gives a **programmable foot stiffness**: the leg can be soft vertically but stiff horizontally, push the ground with a chosen force, and absorb landings. Two cautions: near a **singularity** (leg fully extended, $\det J \rightarrow 0$) the joint torques for a given foot force blow up; and the achievable Cartesian stiffness is again limited by the communication bandwidth of Section 8.5.

7 A Short Control-Theory Primer

This section gives just enough feedback-control background to read the rest.

7.1 Feedback and the PID family

A controller drives the *error* $e = x_d - x$ (desired minus measured) to zero. The classic PID command is

$$u = \underbrace{K_P e}_{\text{stiffness/now}} + \underbrace{K_I \int e dt}_{\text{removes steady error}} + \underbrace{K_D \dot{e}}_{\text{damping/predict}}. \quad (22)$$

- **P** reacts to the present error; larger K_P = stiffer and faster, but too large $\rightarrow$ oscillation.
- **I** accumulates past error to remove steady-state offset (e.g. droop under constant load); too large $\rightarrow$ overshoot / windup.
- **D** reacts to the rate of change; it adds damping and reduces overshoot, but amplifies noise.

Our joint impedance (Eq. (29)) is a **PD** controller in disguise: K_P is the spring, K_D the damper. There is no integral term — desirable for a compliant joint, since I-action would fight external pushes.

7.2 Feedforward

Feedback only acts *after* an error appears. **Feedforward** (FF) adds a command computed from the plan itself — the desired velocity $\dot{q}_d$ in the tracker, or gravity $g(q)$ in the impedance loop — so the system tracks even with zero error. The pattern $FF + PD$ (plan + correction) is used throughout this project and is the practical core of "MIT mode."

7.3 Cascade control

Loops can be nested: an outer loop's output is the inner loop's setpoint (position $\rightarrow$ speed $\rightarrow$ current). Each inner loop must be *faster* than the one outside it. The motor implements this cascade on-board; our trajectory follower adds yet another outer position loop in software on top of the speed loop.

7.4 Stability, delay, and sampling

Three effects erode stability margin and cap usable gains:

- **Loop delay:** the synchronous CAN round-trip (Section 8.5) adds phase lag; the more delay, the lower the K_P you can use before oscillation. This is why torque control is harder here than position control.
- **Sampling:** a discrete loop at rate f_{ctrl} can only act on dynamics well below $\sim f_{\text{ctrl}}/10$. Run as fast as the bus allows.
- **Noise:** differentiating a noisy signal (for a D term) amplifies it; filter velocity if you add damping from measured $\dot{q}$.

Practical tuning recipe: raise K_P until the joint just begins to buzz, back off $\sim 30\%$, then add K_D to remove residual oscillation.

8 Trajectory-Following Control

8.1 Why not "MIT mode"?

High-performance legged actuators (e.g. T-Motor/CubeMars AK) accept a single *impedance* frame

$$\tau = K_p (q_d - q) + K_d (\dot{q}_d - \dot{q}) + \tau_{\text{ff}}, \quad (23)$$

evaluated on the driver at several kHz. **The LK protocol used here does not expose this combined command** — only the separate torque, speed, and position loops. We therefore approximate joint impedance *in software*.

8.2 Velocity feedforward + position PD

Rather than commanding torque, we command *velocity* and let the motor’s internal speed loop realise it. The host computes

$$\dot{\theta}_{\text{cmd}} = \dot{q}_d + K_P (q_d - q_{\text{meas}}) \quad (24)$$

and sends it through `speed_loop_control`. Interpretation:

- $\dot{q}_d$ is the **feedforward**: along the planned path the joint already “knows” how fast to move, so even with zero error it tracks.
- $K_P(q_d - q_{\text{meas}})$ is **proportional position feedback**: it corrects drift and disturbance.

This is an outer position loop whose output is a velocity reference — it *cascades* onto the motor’s speed PI loop. Comparing with Eq. (23), software supplies the position-spring term while the motor’s speed loop supplies the damping/torque generation; a measured-velocity term $-K_D \dot{q}$ can be added later using `spd_raw` from the same reply.

8.3 Trajectory generation

The desired path is produced analytically so that position and its exact derivative are both available. The default is a sinusoidal swing, phase-shifted per joint j :

$$q_{d,j}(t) = q_0 + A \sin(\omega t + \phi_j), \quad (25)$$

$$\dot{q}_{d,j}(t) = A \omega \cos(\omega t + \phi_j), \quad \omega = 2\pi f, \quad \phi_j = j\pi. \quad (26)$$

Using the analytic $\dot{q}_d$ (not a numerical difference) keeps the feedforward clean and noise-free. Any gait curve can replace this as long as it returns the pair $(q_d, \dot{q}_d)$.

8.4 Encoder unwrapping

The single-turn encoder $\theta_{\text{enc}} \in [0, 360)$ wraps. At control rates the per-tick change is small, so we reconstruct a continuous angle by accumulating wrapped differences. With $\Delta = \theta_{\text{enc}}^{(k)} - \theta_{\text{enc}}^{(k-1)}$,

$$\Delta \leftarrow \begin{cases} \Delta - 360, & \Delta > 180, \\ \Delta + 360, & \Delta < -180, \\ \Delta, & \text{otherwise,} \end{cases} \quad \theta_{\text{cont}}^{(k)} = \theta_{\text{cont}}^{(k-1)} + \Delta. \quad (27)$$

The startup reading is taken as the home, so $\theta_{\text{cont}}^{(0)} = 0$ and the trajectory swings about the launch position. (Validity requires $|\dot{\theta}| \Delta t < 180^\circ$, easily satisfied at 200 Hz.)

8.5 Discretisation and loop rate

The controller runs at a fixed period $\Delta t = 1/f_{\text{ctrl}}$ (default 200 Hz). Each tick, for every joint in sequence, it sends one command and waits for the reply. With N joints and round-trip time T_{rt} , the achievable rate is bounded by

$$f_{\text{ctrl}} \leq \frac{1}{N T_{\text{rt}}}. \quad (28)$$

Because the protocol is synchronous and joints are serviced serially, this — not CPU — sets the ceiling. The loop sleeps for the remainder of Δt after its work; if work exceeds Δt the effective period grows. For a full leg or quadruped, non-blocking sends or asynchronous reads are required.

9 Position vs. Torque Control

Sections 4 and 6 commanded *position* or *velocity*: the host says *where* the joint should be and the motor’s inner loops make it stiff. An alternative is **torque control**: the host says *how hard* the joint should push, and stiffness becomes a software choice. This distinction is central for legged robots.

9.1 The trade-off

	Position / velocity control	Torque control
Commanded quantity	where the joint goes	how hard the joint pushes
Joint stiffness	very stiff (rigid)	soft / chosen in software
Ground contact / impact	poor — shock goes to gearbox and frame	good — joint yields and absorbs impact
Foot force $\tau = J^\top \mathbf{F}$	not directly controllable	directly controllable
Backdrivable	no	yes
Ease of use	easy (motor does the work)	hard (needs model + fast loop)
Typical use	arms, free-space tracking	dynamic legged robots

In one line: *position control is for tracking trajectories in free space; torque control is for interacting with the ground.* Walking, jumping, and landing all require compliance and foot-force regulation, which only torque control provides.

9.2 Software joint impedance

Because the LK protocol has no combined impedance frame (Eq. (23)), we evaluate the impedance law on the host and send the result through the *torque* loop (0xA1):

$$\tau = K_P (q_d - q) + K_D (\dot{q}_d - \dot{q}) + \tau_{\text{ff}} \quad (29)$$

The joint then behaves like a **virtual spring-damper** about the desired state: K_P sets stiffness, K_D sets damping, and τ_{ff} is a feedforward (e.g. gravity compensation). Setting $q_d = \text{const}$, $\dot{q}_d = 0$ gives a pure spring that can be pushed by hand and returns softly — the safe way to verify the loop. Unlike the cascade of Section 4, this runs the position/velocity feedback *in software* directly on top of the current loop, so the closed-loop stiffness is exactly K_P (subject to communication bandwidth).

9.3 Torque-to-current conversion

The torque loop accepts a current command $i_q \in [-2048, 2048]$. Using the calibration of Section 3 and the direction sign of Section 3.6, the commanded torque becomes

$$i_q^{\text{cmd}} = \text{clamp}(d \cdot \tau \cdot \text{torque_gain}, -i_q^{\text{max}}, +i_q^{\text{max}}), \quad (30)$$

where the clamp i_q^{max} is a hard safety limit (default 500, i.e. $\approx 500/\text{torque_gain} \approx 2.4 \text{ Nm}$). Always start with low gains and a low clamp: incorrect feedback or excessive K_P can command large forces.

9.4 Gravity feedforward

With a known link mass m , gravity g , and centre-of-mass distance ℓ_c , a single-link gravity term is

$$\tau_{\text{ff}} = m g \ell_c \cos q, \quad (31)$$

so the joint holds its load even with small K_P (genuinely compliant load-holding). For the multi-link leg this generalises to the $g(q)$ vector of the dynamics equation. The code ships this as a stub returning 0 until the leg's parameters are supplied.

9.5 When to use which

- **Free-space trajectory tracking (no contact):** prefer position or velocity-FF + PD — simpler and stable.
- **Contact-rich, dynamic motion (standing soft, force control, jumping/landing):** use torque/impedance control.
- **Hardware caveat:** torque control is far more sensitive to loop latency than position control. The synchronous CAN round-trip (Section 8.5) and absence of a native impedance frame are the real limits here — measure T_{rt} before relying on it.

10 Code Architecture

10.1 File map

File	Role
motor_library.py	LKMotor class: CAN encode/decode, all commands.
config.py	Gains, directions, test angles.
main.py	Single-motor position test + CSV logging.
demo_motors.py	Sequential position demo (ids 1,2), with re-zero.
trajectory_follow.py	Velocity-FF + PD trajectory tracker.
torque_impedance.py	Software joint impedance via the torque loop.

10.2 The LKMotor class

Construction opens the bus and zeroes the cached state:

```
1 m = LKMotor(bus_interface="socketcan", bus_channel="can0",  
2             motor_id=1, bitrate=1_000_000)
```

Internally:

- `_send_command(cmd, data)` builds the 8-byte frame at `0x140+motor_id` and transmits it;
- `_receive_response(timeout)` blocks until a reply with the matching id arrives (or times out $\rightarrow$ None);
- `_parse_response_1/2/3` decode the status payloads into the cached fields and return them as a tuple.

10.3 Walkthrough: trajectory_follow.py

The control law of Eq. (24), the unwrap of Section 8.4, and the sign convention of Section 3.6 appear directly in the inner loop:

```
1 q_des, qd_des = trajectory(t, ji)           # generated path (deg, deg/s)  
2 q_meas = cont_deg[mid] * d                  # unwrapped, sign-corrected  
3  
4 err = q_des - q_meas                        # position error
```

```

5 speed_cmd = qd_des + KP * err          # velocity FF + P (deg/s)
6
7 res = motors[mid].speed_loop_control(  # command via inner speed loop
8     IQ_LIMIT,
9     int(speed_cmd * d * param.vel_gain))
10
11 _, _, _, enc = res                    # encoder feedback (single-turn)
12 raw = enc * param.encoder_gain        # 0..360 deg
13 delta = raw - prev_raw[mid]          # unwrap ...
14 if delta > 180: delta -= 360
15 elif delta < -180: delta += 360
16 cont_deg[mid] += delta
17 prev_raw[mid] = raw

```

On exit the loop commands zero speed and releases every motor, so the leg does not run away on Ctrl-C.

10.4 Walkthrough: torque_impedance.py

The impedance law of Eq. (29) and the torque-to-current conversion appear in the inner loop; note that velocity feedback $\dot{q}$ is now used too:

```

1 q_des, qd_des = trajectory(t, ji)      # const home (spring) or sinusoid
2 q_meas = cont_deg[mid] * d             # unwrapped position
3 qd_now = qd_meas[mid]                  # measured velocity (from last reply)
4
5 tau = (KP * (q_des - q_meas)           # spring
6       + KD * (qd_des - qd_now)        # damper
7       + gravity_feedforward(q_meas, ji)) # feedforward (stub -> 0)
8
9 iq_cmd = clamp(int(tau * param.torque_gain * d), -IQ_MAX, IQ_MAX)
10 res = motors[mid].torque_loop_control(iq_cmd)
11
12 _, iq_raw, spd_raw, enc = res
13 qd_meas[mid] = d * spd_raw / param.vel_state_gain # update velocity
14 # ... encoder unwrap as before ...

```

On exit the loop sends `torque_loop_control(0)` before releasing, so the joints go limp rather than holding force.

11 Practical Notes, Limitations, and Tuning

- **Gain K_P :** start small. Too large $\rightarrow$ buzzing / instability (the cascade plus communication delay reduces phase margin); too small $\rightarrow$ sluggish tracking.
- **No native impedance:** stiffness/damping at the joint are limited by what the speed loop and software K_P provide; true variable impedance would need the torque loop plus the model of Section 5.
- **Single-turn absolute encoder:** multi-turn position is only known after software unwrapping from a known home; a power cycle loses the continuous count.
- **set_position_to_angle (0x95)** re-zeros in RAM (no wear, lost on power cycle); **set_zero_position (0x19)** writes flash and persists — use sparingly.
- **Communication ceiling:** see Section 8.5; measure the true loop period on hardware before trusting timing.
- **Lost packets:** every command may return `None`; the controllers treat this as a skipped tick.

12 Suggested Next Steps

A roughly ordered path from the current working code toward a controllable leg:

1. **Data logging & plots.** Log $q_d, q, \dot{q}, \tau$ to CSV and plot tracking error vs. time. Tune gains from data, not feel. (No hardware parameters needed — do this first.)
2. **Gravity compensation.** Fill the $\tau_{\text{ff}} = mgl_c \cos q$ stub so joints hold load with low stiffness. Needs link masses, lengths, and centre-of-mass positions.
3. **Safety layer.** Joint-angle limits, a torque soft-start ramp, and a lost-packet watchdog that goes limp on comms loss — before pushing gains or attempting dynamic motion.
4. **Cartesian foot control.** Implement FK + Jacobian and $\tau = J^T \mathbf{F} + g(q)$ for programmable foot stiffness and force. Needs link lengths and a defined joint-zero convention. This is the headline legged-robot capability.
5. **Measure the loop rate.** Time the CAN round-trip T_{rt} on hardware; it determines whether high-bandwidth torque control is viable with this protocol.

A Symbol Reference

Symbol	Meaning
$q, \dot{q}$	joint angle, angular velocity (output shaft)
$q_d, \dot{q}_d$	desired joint angle / velocity
$\theta_{\text{out}}, \theta_{\text{motor}}$	output- / motor-shaft angle
$r_g = 0.1$	gear ratio (output/motor)
$d = \pm 1$	joint direction multiplier
i_q	quadrature (torque-producing) current
K_t	motor torque constant
K_P	software position-feedback gain
$J(q)$	foot Jacobian
$\Delta t, f_{\text{ctrl}}$	control period / rate

B Glossary

Term	Meaning
CAN	Controller Area Network — the serial bus protocol
SocketCAN	Linux CAN networking stack
LSB	least-significant bit — the raw integer step of a quantity
BLDC / PMSM	brushless DC / permanent-magnet synchronous motor
FOC	field-oriented control (current control in the rotor frame)
i_q, i_d	quadrature (torque) / direct current components
PD / PID	proportional-derivative / -integral-derivative control
FF	feedforward (command from the plan, not the error)
FK / IK	forward / inverse kinematics
Jacobian	matrix mapping joint rates to foot velocity
dps	degrees per second
impedance	relationship between motion and force (stiffness + damping)
backdrivable	able to be moved by external force